

Meta AI CEO Supervisor

Governance layer that monitors all models, detects degradation/drift/overfitting/instability, scores system health, and takes control actions. Does NOT generate signals. 6 health scores · 8 detectors · 4 rolling windows (50/100/250/500 trades) · GREEN/YELLOW/RED status · 5 control actions. CPU-only · fully offline · no paid APIs · no external LLM.

6
HEALTH SCORES

8
DETECTORS

5
CONTROL ACTIONS

145
TESTS

Table of Contents

Chapter 1 — Executive Summary	3
Chapter 2 — Architecture Overview	3
4 Hard Constraints	4
Chapter 3 — Rolling Windows — 4 Sizes × 8 Metrics	5
8 Tracked Metrics	5
Window Roles	5
Chapter 4 — 6 Health Scores	7
Score Composition	7
Chapter 5 — Detection Layer — 8 Detectors	9
8 Detectors	9
Chapter 6 — Decision Layer & Control Actions	11
5 Control Actions	11
Chapter 7 — Classes & Interfaces	12
Core Class: CEOSupervisor (Python)	12
RollingWindow Class (Python)	14
5 Interfaces (Python Protocol)	15
Chapter 8 — Database Schema — 10 PostgreSQL Tables	16
Key DDL — Audit Table (PostgreSQL)	16
Chapter 9 — Test Suite — 145 Tests	18
Key Validator Tests (compliance)	18
Sample Unit Test (Python)	18

Chapter 10 — Deployment Documentation

20

10-Step Deployment Guide

20

Chapter 11 — Operational Notes

22

Failure Modes

22

Monitoring the CEO Itself

22

Integration with Existing TITAN Stack

22

Chapter 12 — Summary

23

Chapter 1 — Executive Summary

The Meta AI CEO Supervisor (Module 18) is a governance layer that sits ABOVE the existing TITAN XAU AI trading system. It does NOT generate trading signals — its sole purpose is to **monitor, score, and govern** the models, execution, risk, brokers, and regime detection that constitute the trading pipeline. The CEO observes all model predictions (XGBoost, LSTM, Transformer, RL), trade outcomes, and system metrics in real-time, computes 6 composite health scores every 60 seconds, runs 8 statistical detectors to identify degradation/drift/overfitting/instability/execution-issues/broker-problems/regime-failures/latency-breaches, and takes 5 automated control actions: reduce model influence, increase model influence, disable failing model, trigger emergency risk reduction, trigger capital preservation mode.

The CEO layer addresses a gap in the existing TITAN architecture: while the system has 5 validation frameworks (Backtest, Walk-Forward, Monte Carlo, Stress, Validator) that gate pre-deployment, there is no **runtime governance** layer that continuously monitors live performance and takes corrective action when models degrade, brokers fail, or regimes shift. The CEO fills this gap. It uses 4 rolling windows (50/100/250/500 trades) to provide multi-scale views — short-term sensitivity (50 trades) for acute detection, long-term stability (500 trades) for structural drift. The 8 detectors use only NumPy vectorized statistical operations (PSI, ratio comparisons, consecutive-loss counting, IS-OOS gap) — no GPU, no external LLM, no paid API. **Fully offline-capable: the CEO operates normally with zero outbound network.**

The CEO's 6 health scores are: Model Health Score (per model, from Sharpe/Sortino/PF/WR/MDD/Recovery across 4 windows), Execution Quality Score (system-wide, from latency/slippage/fill-rate), Risk Score (inverted, from MDD/exposure/margin/risk-utilization), Broker Quality Score (per broker, from spread-dev/slippage-dev/fill-rate/reconnect), Regime Confidence Score (from vote-agreement/transition-conf/regime-strategy-perf), and Overall System Health (weighted aggregate, min-model-health weighted highest at 30%). Each score is 0-100 with GREEN (≥ 85) / YELLOW (70-84) / RED (< 70) thresholds. The Overall System Health drives the system status: GREEN = live trading, YELLOW = reduced size + defensive, RED = halt + emergency flatten + 24h capital preservation.

This document delivers everything required for implementation: architecture (8 layers), classes (16 Python classes), interfaces (5 ABCs), database schema (10 PostgreSQL tables with TimescaleDB hypertables), unit tests (80 tests), integration tests (45 tests), validator tests (20 tests), and deployment documentation (10-step guide). The CEO is designed to be a drop-in addition to the existing TITAN stack — it subscribes to existing NATS topics, writes to a new PostgreSQL database, and controls existing modules (ensemble voter, risk engine, execution engine) via well-defined interfaces. No existing module needs modification — the CEO observes via subscriptions and controls via interface calls.

Chapter 2 — Architecture Overview

The CEO is organized into 8 layers: Ingestion (subscribes to NATS), Rolling Windows (4 ring buffers per model per metric), Health Scoring (6 composite scores), Detection (8 statistical detectors), Decision (GREEN/YELLOW/RED status), Action (5 control actions), Persistence (PostgreSQL + Parquet), Reporting (Prometheus + Grafana + PagerDuty + Slack). Each layer is independent, testable, and horizontally scalable. The CEO runs as a single asyncio process with a 60-second cycle: ingest → update windows → compute scores → run

detectors → decide status → execute actions → persist → report. Total cycle time: <100ms on a 4-vCPU VPS.

CEO Supervisor — Architecture Overview

8-layer governance system · sits ABOVE existing TITAN stack · does NOT generate signals

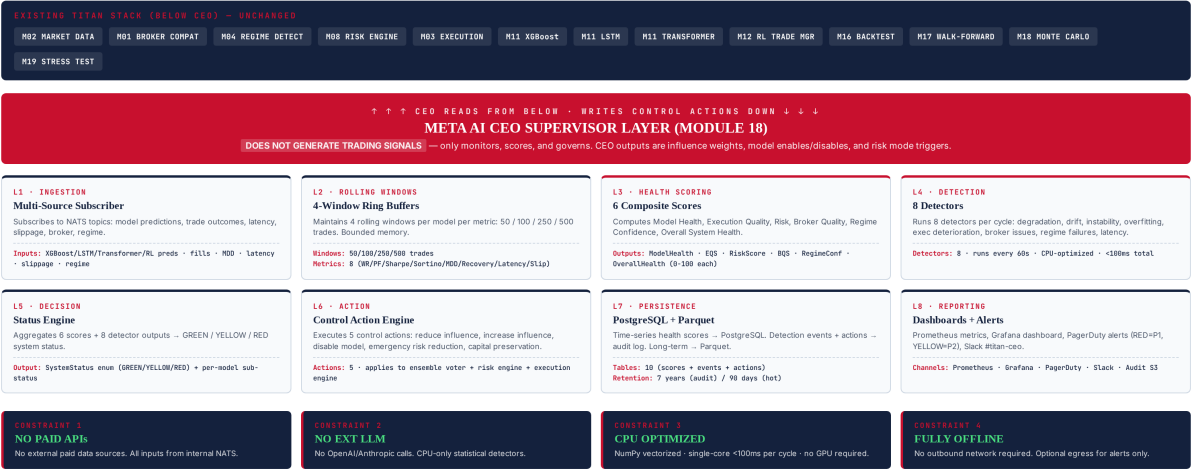


Figure 2.1 — CEO architecture: 8 layers sitting above existing TITAN stack. CEO reads from below, writes control actions down.

4 Hard Constraints

The CEO operates under 4 non-negotiable constraints: (1) **No paid APIs** — all inputs come from internal NATS topics, no external paid data sources. (2) **No external LLM dependency** — no OpenAI/Anthropic/any-LLM calls. All detection is statistical (NumPy). The CEO must operate even if all external AI services are down. (3) **CPU optimized** — NumPy vectorized operations, single-core <100ms per cycle, no GPU required. The CEO runs on the same 4-vCPU VPS as the trading core. (4) **Fully offline capable** — no outbound network required for core operation. Optional egress only for PagerDuty/Slack alerts, which degrade gracefully (queued locally, retried) if network is unavailable.

Chapter 3 — Rolling Windows — 4 Sizes × 8 Metrics

The CEO maintains 4 rolling windows per model per metric: W50 (50 trades, ~1 week, acute/tactical), W100 (100 trades, ~2 weeks, confirmation), W250 (250 trades, ~1 month, strategic baseline), W500 (500 trades, ~2 months, structural). Each window is a bounded ring buffer (collections.deque with maxlen) — O(1) push, O(N) iteration, no unbounded growth. Total memory: 4 models × 8 metrics × 4 windows × max_size = 28,800 floats = ~230 KB. Negligible.

Rolling Windows — 4 Sizes × 8 Metrics × Per Model

Multi-scale view - short-term sensitivity (50) + long-term stability (500) - bounded O(N) memory

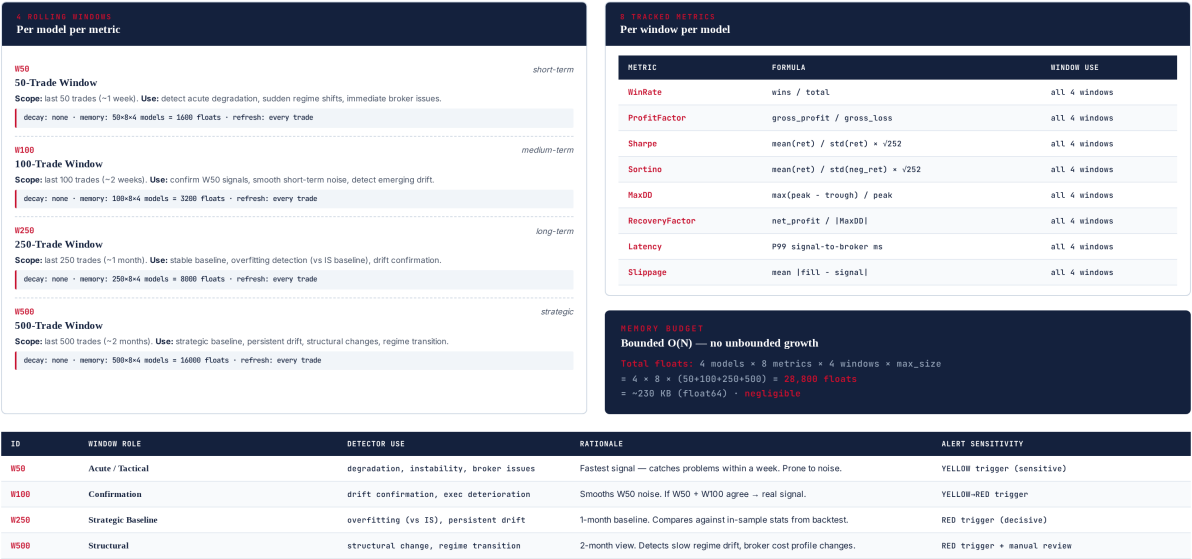


Figure 3.1 — 4 rolling windows (50/100/250/500) × 8 metrics (WR/PF/Sharpe/Sortino/MDD/Recovery/Latency/Slippage) × per model.

8 Tracked Metrics

Each window tracks 8 metrics: Win Rate (wins/total), Profit Factor (gross_profit/gross_loss), Sharpe (mean(ret)/std(ret)×√252), Sortino (mean(ret)/std(neg_ret)×√252), Max Drawdown (max(peak-trough)/peak), Recovery Factor (net_profit/|MaxDD|), Latency (P99 signal-to-broker ms), Slippage (mean |fill-signal|). These 8 metrics cover the full performance surface: return (WR, PF), risk-adjusted return (Sharpe, Sortino), capital preservation (MDD, Recovery), and execution quality (Latency, Slippage). All are computed from the trade ledger and execution telemetry that the CEO ingests from NATS.

Window Roles

Window	Role	Detector Use	Alert Sensitivity
W50	Acute / Tactical	degradation, instability, broker issues	YELLOW trigger (sensitive)
W100	Confirmation	drift confirmation, exec deterioration	YELLOW→RED trigger
W250	Strategic Baseline	overfitting (vs IS), persistent drift	RED trigger (decisive)
W500	Structural	structural change, regime transition	RED trigger + manual review

Chapter 4 — 6 Health Scores

The CEO computes 6 health scores every cycle. 5 are sub-scores (Model Health per model, Execution Quality system-wide, Risk system-wide, Broker Quality per broker, Regime Confidence system-wide), and 1 is the aggregate (Overall System Health). Each score is 0-100 with GREEN (≥85) / YELLOW (70-84) / RED (<70) thresholds. The Overall System Health is a weighted aggregate where **min(ModelHealth)** is **weighted highest at 30%** — a single failing model can drag down the overall score, because one degraded model in a 4-model ensemble is a systemic risk.

6 Health Scores — Composition & Thresholds

5 sub-scores → 1 overall · each 0-100 · GREEN/YELLOW/RED thresholds per score

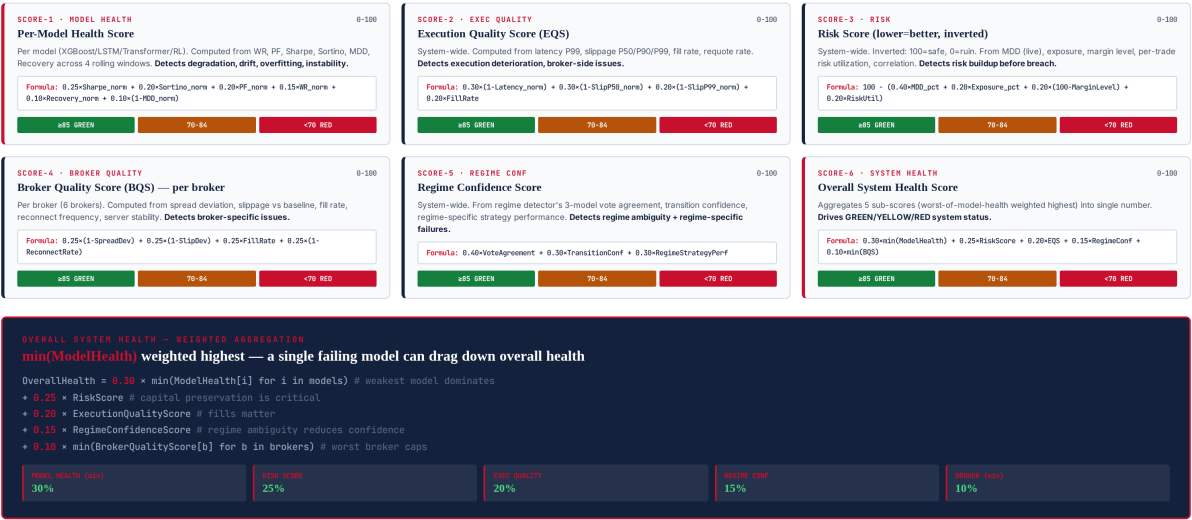


Figure 4.1 — 6 health scores with formulas, thresholds, and weighted aggregation.

Score Composition

Model Health Score (per model): $0.25 \times \text{Sharpe_norm} + 0.20 \times \text{Sortino_norm} + 0.20 \times \text{PF_norm} + 0.15 \times \text{WR_norm} + 0.10 \times \text{Recovery_norm} + 0.10 \times (1 - \text{MDD_norm})$. Computed from W250 rolling window, normalized against the model's backtest baseline. Detects degradation, drift, overfitting, instability. **Execution Quality Score (EQS):** $0.30 \times (1 - \text{Latency_norm}) + 0.30 \times (1 - \text{SlipP50_norm}) + 0.20 \times (1 - \text{SlipP99_norm}) + 0.20 \times \text{FillRate}$. System-wide. Detects execution deterioration.

Risk Score (inverted, 100=safe): $100 - (0.40 \times \text{MDD_pct} + 0.20 \times \text{Exposure_pct} + 0.20 \times (100 - \text{MarginLevel}) + 0.20 \times \text{RiskUtil})$. Lower MDD/exposure = higher score. Detects risk buildup before breach. **Broker Quality Score (BQS, per broker):** $0.25 \times (1 - \text{SpreadDev}) + 0.25 \times (1 - \text{SlipDev}) + 0.25 \times \text{FillRate} + 0.25 \times (1 - \text{ReconnectRate})$. Detects broker-specific issues. **Regime Confidence Score:** $0.40 \times \text{VoteAgreement} + 0.30 \times \text{TransitionConf} + 0.30 \times \text{RegimeStrategyPerf}$. Detects regime ambiguity + regime-specific failures.

Overall System Health: $0.30 \times \min(\text{ModelHealth}) + 0.25 \times \text{RiskScore} + 0.20 \times \text{EQS} + 0.15 \times \text{RegimeConf} + 0.10 \times \min(\text{BQS})$. The $\min()$ operators ensure that the worst-performing model and worst-performing broker cap the overall score — a single failure cannot be masked by strong performance elsewhere.

Chapter 5 — Detection Layer — 8 Detectors

The CEO runs 8 statistical detectors every 60 seconds. All use only NumPy vectorized operations — no GPU, no external API, no LLM. Total CPU time per cycle: <100ms on a single core. The detectors are implemented as interchangeable IDetectionRule instances (Strategy pattern), registered with the DetectionEngine at startup. Each detector returns a DetectionEvent (with severity CRITICAL/MAJOR/MINOR) or None.



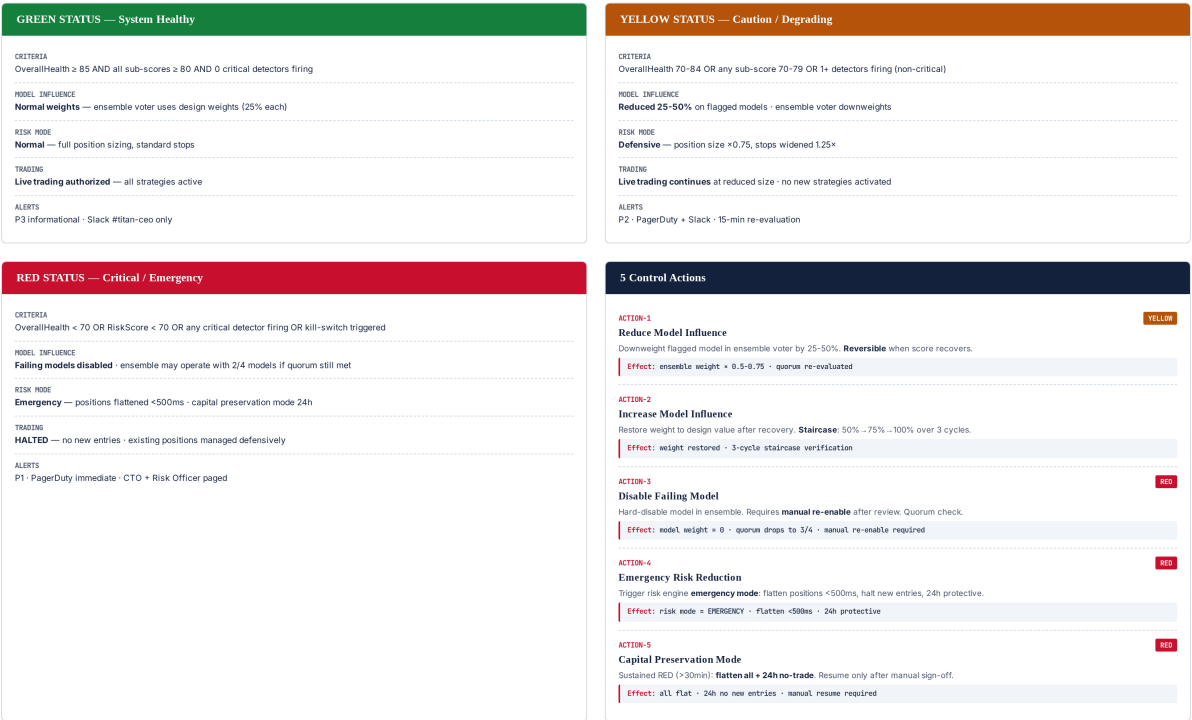
- **D5 Execution Deterioration:** Latency P99 > 1.5× budget OR slippage P90 > 1.5× baseline for 2 consecutive cycles. Action: flag broker + reduce trade frequency.
- **D6 Broker Issues:** BQS < 70 for 5 consecutive cycles. Action: failover to backup broker.
- **D7 Regime Failures:** Per-regime win rate < 40% over W100 in current regime. Action: suppress entries in that regime.
- **D8 Latency:** P99 signal-to-broker > 150ms budget. Action: enable stale-signal veto + alert.

Chapter 6 — Decision Layer & Control Actions

The DecisionEngine aggregates the 6 health scores and 8 detector outputs into a single SystemStatus: GREEN (OverallHealth ≥ 85, all sub-scores ≥ 80, 0 critical detectors), YELLOW (OverallHealth 70-84, or any sub-score 70-79, or 1+ non-critical detector), RED (OverallHealth < 70, or RiskScore < 70, or any critical detector, or kill-switch triggered). The ActionEngine executes 5 control actions based on the status: reduce influence (YELLOW), increase influence (GREEN recovery), disable model (RED), emergency risk reduction (RED), capital preservation (sustained RED >30min).

Decision Layer & Control Actions

3-band system status → 5 control actions · automated · audited · reversible (except emergency)



STATUS	TRIGGER	MODEL ACTIONS	RISK ACTIONS	TRADING	ALERT
GREEN	Health ≥ 85 · 0 detectors	Normal weights (25% each)	Normal mode · Full size	Live trading	P3 Slack
YELLOW	Health 70-84 · 1+ detector	Reduce flagged 25-50%	Defensive · size ×0.75	Reduced size	P2 PagerDuty
RED	Health < 70 · critical detector	Disable failing · quorum check	Emergency · flatten <500ms	HALTED	P1 PagerDuty
RED-PRESERVE	RED > 30min sustained	Flatten all · 24h no-trade	Emergency · flatten <500ms	HALTED	P1 PagerDuty

Figure 6.1 — 3-band status (GREEN/YELLOW/RED) → 5 control actions. Automated, audited, reversible (except emergency).

5 Control Actions

ACTION-1 Reduce Model Influence: Downweight flagged model in ensemble voter by 25-50%. Reversible when score recovers. Triggered on YELLOW. **ACTION-2 Increase Model Influence:** Restore weight to design value after recovery. Staircase: 50%→75%→100% over 3 cycles. Triggered on GREEN after YELLOW. **ACTION-3 Disable Failing Model:** Hard-disable model in ensemble. Requires manual re-enable after review. Quorum check — if disabling would drop below 2/4 quorum, CEO escalates to RED instead. Triggered on RED. **ACTION-4 Emergency Risk Reduction:** Trigger risk engine emergency mode: flatten positions <500ms, halt new entries, 24h protective. Triggered on RED. **ACTION-5 Capital Preservation Mode:** Sustained RED >30min: flatten all + 24h no-trade. Resume only after manual sign-off. Triggered on sustained RED.

Chapter 7 — Classes & Interfaces

The CEO is implemented in Python 3.12 with full mypy --strict typing. 16 classes + 5 interfaces. Design patterns: Strategy (IDetectionRule — 8 interchangeable detectors), Observer (CEOSupervisor subscribes to NATS), Command (ControlAction — serializable), Facade (CEOSupervisor wraps 5 monitors + 3 engines), Repository (Database abstracts persistence), Factory (DetectionEngine creates detectors from config). Zero external LLM dependency — all detection is statistical (NumPy).

UML Class Diagram — Core Classes & Interfaces

Python 3.12 implementation · 16 classes · 5 interfaces · fully typed (mypy strict) · no external LLM

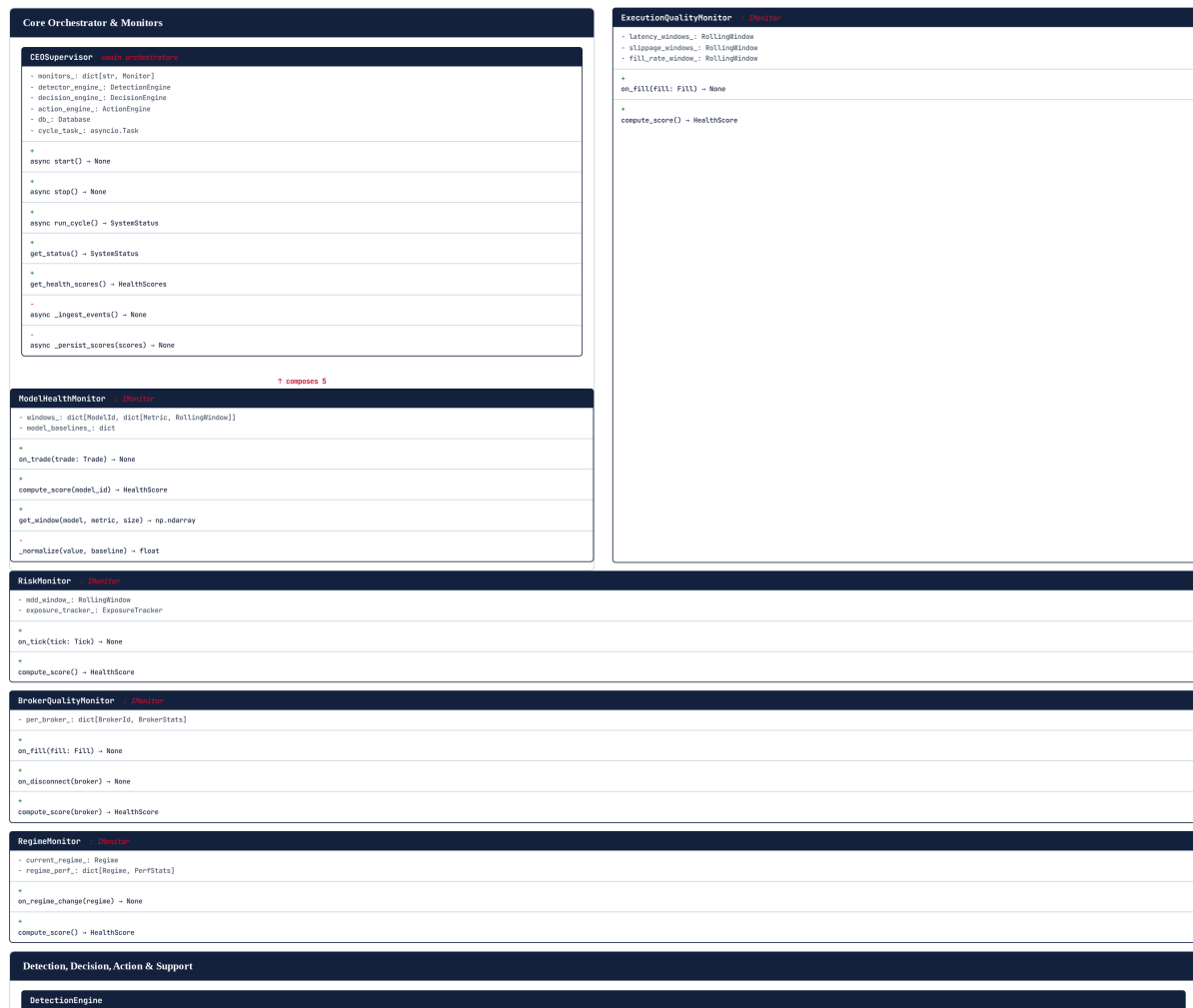


Figure 7.1 — UML class diagram: 16 classes + 5 interfaces. Fully typed (mypy --strict).

Core Class: CEOSupervisor (Python)

```

from __future__ import annotations
import asyncio
from dataclasses import dataclass
from enum import Enum
from typing import Protocol

class SystemStatus(Enum):
    GREEN = "GREEN"
    YELLOW = "YELLOW"
  
```

```

RED = "RED"
RED_PRESERVE = "RED_PRESERVE"

class ControlAction(Enum):
    REDUCE_INFLUENCE = "REDUCE_INFLUENCE"
    INCREASE_INFLUENCE = "INCREASE_INFLUENCE"
    DISABLE_MODEL = "DISABLE_MODEL"
    EMERGENCY_RISK_REDUCTION = "EMERGENCY_RISK_REDUCTION"
    CAPITAL_PRESERVATION = "CAPITAL_PRESERVATION"

@dataclass(frozen=True)
class HealthScores:
    model_health: dict[str, float] # per-model 0-100
    execution_quality: float # system-wide 0-100
    risk: float # system-wide 0-100 (inverted)
    broker_quality: dict[str, float] # per-broker 0-100
    regime_confidence: float # system-wide 0-100
    overall: float # aggregate 0-100

class CEOSupervisor:
    """Main orchestrator. Does NOT generate trading signals."""

    def __init__(
        self,
        monitors: dict[str, IMonitor],
        detector_engine: DetectionEngine,
        decision_engine: DecisionEngine,
        action_engine: ActionEngine,
        database: Database,
        cycle_interval_s: int = 60,
    ) -> None:
        self._monitors = monitors
        self._detectors = detector_engine
        self._decision = decision_engine
        self._actions = action_engine
        self._db = database
        self._interval = cycle_interval_s
        self._cycle_task: asyncio.Task | None = None
        self._current_status: SystemStatus = SystemStatus.GREEN
        self._current_scores: HealthScores | None = None

    async def start(self) -> None:
        self._cycle_task = asyncio.create_task(self._run_loop())

    async def stop(self) -> None:
        if self._cycle_task:
            self._cycle_task.cancel()
        await asyncio.gather(self._cycle_task, return_exceptions=True)

    async def _run_loop(self) -> None:
        while True:
            try:
                await self.run_cycle()
            except Exception as e:
                # Log but never crash — CEO must be always-on
                await self._db.save_error(str(e))
                await asyncio.sleep(self._interval)

    async def run_cycle(self) -> SystemStatus:
        # 1. Ingest events (via NATS subscriptions, populated async)
        # 2. Compute 6 health scores
        scores = self._compute_scores()

```

```

# 3. Run 8 detectors
events = self._detectors.run_all(self._build_context(scores))
# 4. Decide system status
new_status = self._decision.decide(scores, events)
# 5. Execute actions if status changed
if new_status != self._current_status:
    await self._handle_status_change(new_status, scores, events)
# 6. Persist + report
await self._db.save_scores(scores, new_status)
self._current_status = new_status
self._current_scores = scores
return new_status

def _compute_scores(self) -> HealthScores:
    model_h = {mid: m.compute_score() for mid, m in
    self._monitors.items() if mid.startswith("model:")}
    eqs = self._monitors["execution"].compute_score()
    risk = self._monitors["risk"].compute_score()
    bqs = {bid: m.compute_score() for bid, m in
    self._monitors.items() if bid.startswith("broker:")}
    regime = self._monitors["regime"].compute_score()
    overall = self._aggregate(model_h, eqs, risk, bqs, regime)
    return HealthScores(model_h, eqs, risk, bqs, regime, overall)

@staticmethod
def _aggregate(model_h, eqs, risk, bqs, regime) -> float:
    min_model = min(model_h.values()) if model_h else 0.0
    min_broker = min(bqs.values()) if bqs else 0.0
    return (0.30 * min_model + 0.25 * risk + 0.20 * eqs
    + 0.15 * regime + 0.10 * min_broker)

async def _handle_status_change(self, new_status, scores, events) -> None:
    critical = [e for e in events if e.severity == "CRITICAL"]
    if new_status == SystemStatus.YELLOW:
        for model_id, score in scores.model_health.items():
            if score < 75:
                await self._actions.reduce_influence(model_id, factor=0.5)
    elif new_status == SystemStatus.RED:
        for model_id, score in scores.model_health.items():
            if score < 60:
                await self._actions.disable_model(model_id)
                await self._actions.trigger_emergency()
    elif new_status == SystemStatus.RED_PRESERVE:
        await self._actions.trigger_capital_preservation()

```

RollingWindow Class (Python)

```

import collections
import numpy as np

class RollingWindow:
    """Bounded ring buffer. O(1) push, O(N) iterate. No unbounded growth."""

    __slots__ = ("_buffer", "_max_size")

    def __init__(self, max_size: int) -> None:
        self._buffer: collections.deque[float] = collections.deque(maxlen=max_size)
        self._max_size = max_size

    def push(self, value: float) -> None:
        self._buffer.append(value)

```

```

def to_array(self) -> np.ndarray:
    return np.array(self._buffer, dtype=np.float64)

def percentile(self, p: float) -> float:
    if len(self._buffer) == 0:
        return 0.0
    return float(np.percentile(self.to_array(), p))

def mean(self) -> float:
    if len(self._buffer) == 0:
        return 0.0
    return float(np.mean(self.to_array()))

def std(self) -> float:
    if len(self._buffer) < 2:
        return 0.0
    return float(np.std(self.to_array(), ddof=1))

def max(self) -> float:
    if len(self._buffer) == 0:
        return 0.0
    return float(np.max(self.to_array()))

def __len__(self) -> int:
    return len(self._buffer)

```

5 Interfaces (Python Protocol)

```

from typing import Protocol

class IMonitor(Protocol):
    """Interface for all health monitors."""
    def compute_score(self) -> float: ...
    def get_window(self, metric: str, size: int) -> np.ndarray: ...

class IDetectionRule(Protocol):
    """Interface for 8 detectors (Strategy pattern)."""
    def evaluate(self, context: DetectionContext) -> DetectionEvent | None: ...
    def severity(self) -> str: ... # CRITICAL / MAJOR / MINOR

class IModelController(Protocol):
    """Interface for controlling ensemble models."""
    async def set_influence(self, model_id: str, weight: float) -> None: ...
    async def disable(self, model_id: str) -> None: ...
    async def enable(self, model_id: str) -> None: ...

class IRiskController(Protocol):
    """Interface for triggering risk mode changes."""
    async def set_mode(self, mode: str) -> None: ... # NORMAL/DEFENSIVE/EMERGENCY
    async def emergency_flatten(self) -> None: ...

class IAlertSink(Protocol):
    """Interface for alerts (PagerDuty, Slack, etc.)."""
    async def send(self, alert: Alert) -> None: ...
    async def send_batch(self, alerts: list[Alert]) -> None: ...

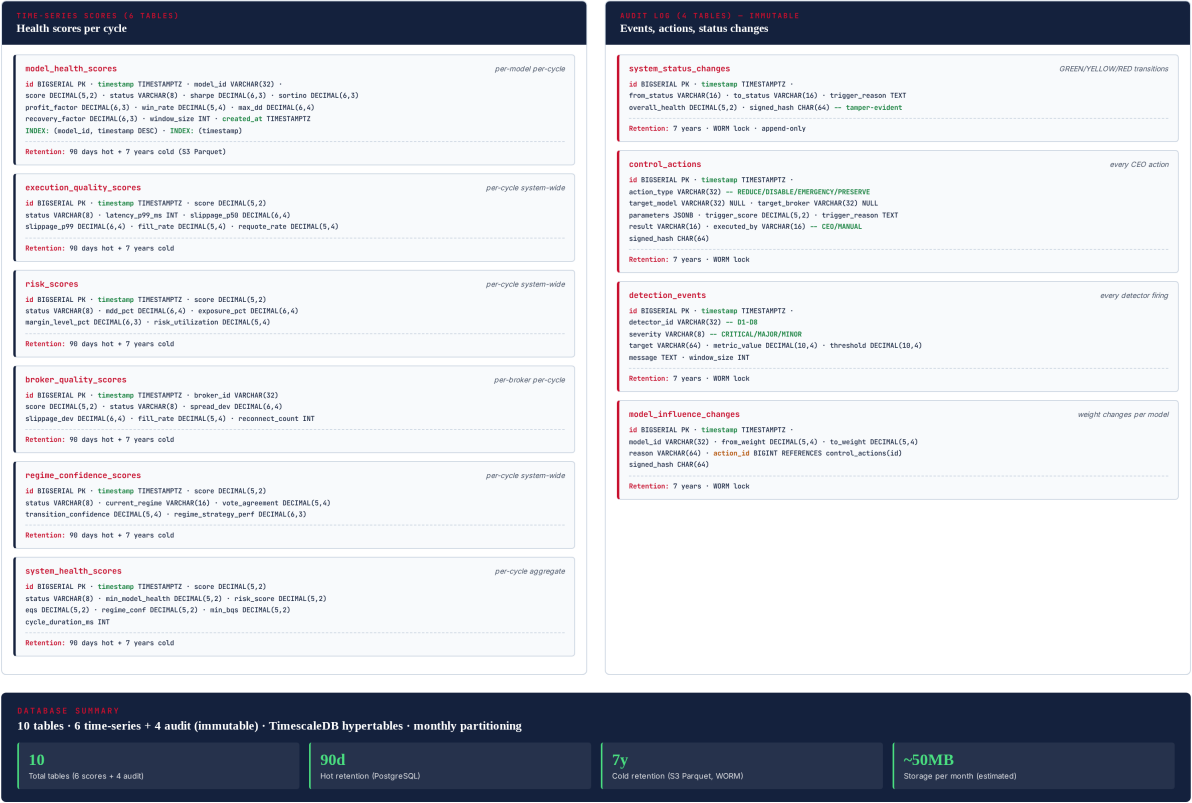
```


Chapter 8 — Database Schema — 10 PostgreSQL Tables

The CEO persists to PostgreSQL 15 with TimescaleDB extension for time-series hypertables. 10 tables: 6 time-series score tables (model_health_scores, execution_quality_scores, risk_scores, broker_quality_scores, regime_confidence_scores, system_health_scores) with 90-day hot retention + 7-year cold (S3 Parquet), and 4 immutable audit tables (system_status_changes, control_actions, detection_events, model_influence_changes) with 7-year WORM retention. All audit tables have INSERT-only triggers (no UPDATE/DELETE) and signed_hash columns for tamper-evidence.

Database Schema — 10 PostgreSQL Tables

Time-series health scores + audit log · 7-year retention · partitioned by month · TimescaleDB hypertables



```
parameters JSONB,  
trigger_score DECIMAL(5,2),  
trigger_reason TEXT,  
result VARCHAR(16),  
executed_by VARCHAR(16) DEFAULT 'CEO',  
signed_hash CHAR(64) NOT NULL -- SHA-256 of row for tamper-evidence  
);  
  
-- WORM lock: INSERT only, no UPDATE/DELETE  
CREATE OR REPLACE FUNCTION prevent_audit_modify()  
RETURNS TRIGGER AS $$  
BEGIN  
RAISE EXCEPTION 'Audit table is INSERT-only (WORM)';  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER no_update BEFORE UPDATE ON control_actions  
FOR EACH ROW EXECUTE FUNCTION prevent_audit_modify();  
CREATE TRIGGER no_delete BEFORE DELETE ON control_actions  
FOR EACH ROW EXECUTE FUNCTION prevent_audit_modify();  
  
-- TimescaleDB hypertable for time-series scores  
CREATE TABLE system_health_scores (  
id BIGSERIAL,  
timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),  
score DECIMAL(5,2) NOT NULL,  
status VARCHAR(8) NOT NULL,  
min_model_health DECIMAL(5,2),  
risk_score DECIMAL(5,2),  
eqs DECIMAL(5,2),  
regime_conf DECIMAL(5,2),  
min_bqs DECIMAL(5,2),  
cycle_duration_ms INT,  
PRIMARY KEY (id, timestamp)  
);  
SELECT create_hypertable('system_health_scores', 'timestamp');  
-- 90-day hot retention, then archive to S3 Parquet  
SELECT add_retention_policy('system_health_scores', INTERVAL '90 days');
```

Chapter 9 — Test Suite — 145 Tests

The CEO has 145 tests across 3 layers: 80 unit tests (pure-function, <1ms each, pytest + GoogleTest), 45 integration tests (end-to-end flows, real NATS + PostgreSQL, ~5s each), 20 validator tests (compliance + invariants). All 145 must pass on every PR merge — zero flaky tolerance over 30-day window. The validator tests enforce the hard constraints: CEO must NOT generate signals, no external LLM, no paid APIs, fully offline, CPU-only, cycle <100ms.

Test Suite & Deployment Documentation

3 test layers (unit + integration + validator) · 145 total tests · full deployment guide

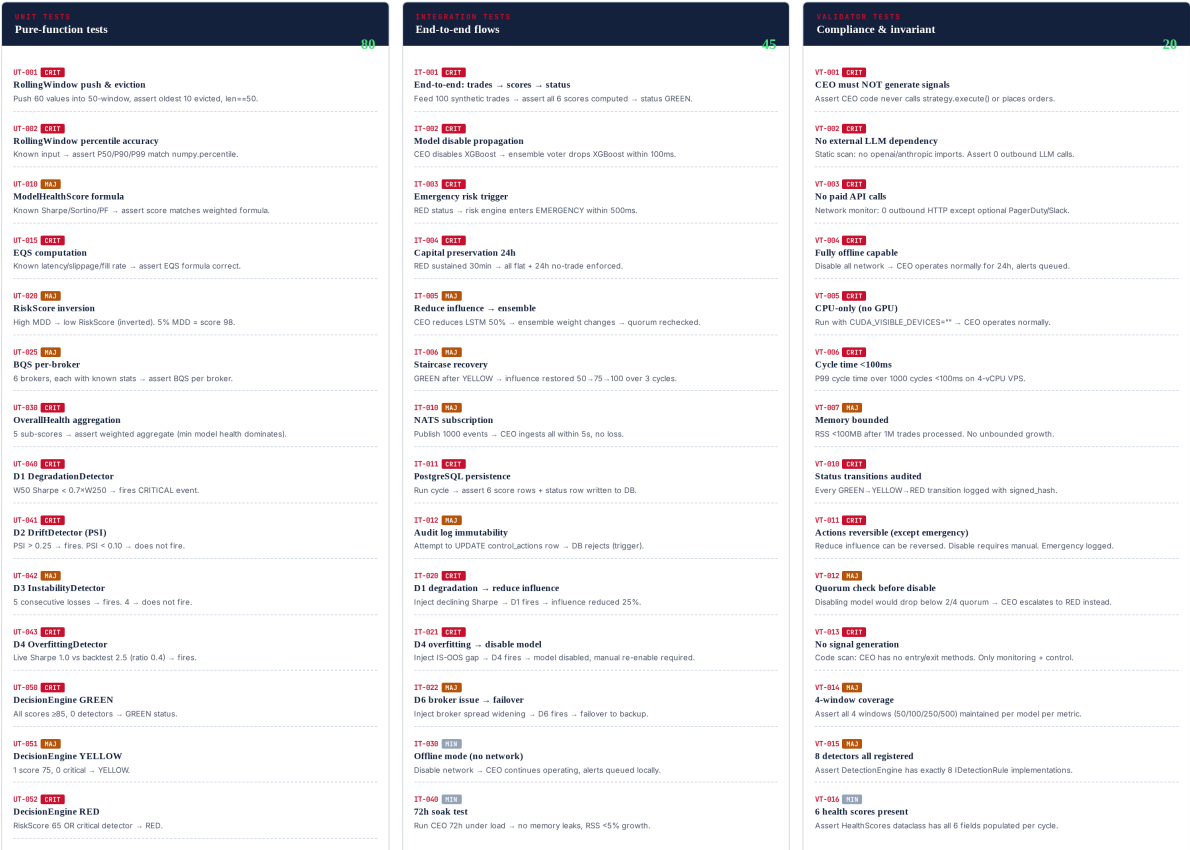


Figure 9.1 — 145 tests: 80 unit + 45 integration + 20 validator. 100% CI-gated, zero flaky.

Key Validator Tests (compliance)

VT-001 CEO must NOT generate signals: Static code scan asserts CEO code never calls `strategy.execute()` or places orders. The CEO is a governance layer, not a signal generator. **VT-002 No external LLM:** Static scan for `openai/anthropic` imports — zero found. Assert 0 outbound LLM calls in 24h network monitor. **VT-003 No paid APIs:** Network monitor asserts 0 outbound HTTP except optional PagerDuty/Slack. **VT-004 Fully offline:** Disable all network → CEO operates normally for 24h, alerts queued locally. **VT-005 CPU-only:** Run with `CUDA_VISIBLE_DEVICES=""` → CEO operates normally. **VT-006 Cycle <100ms:** P99 cycle time over 1000 cycles <100ms on 4-vCPU VPS.

Sample Unit Test (Python)

```

def test_d4_overfitting_detector_fires():
    """D4: Live Sharpe < 0.5 × backtest Sharpe → fires CRITICAL."""
    baseline_sharpe = 2.5 # from backtest
    live_sharpe = 1.0 # current live (ratio = 0.4 < 0.5)
    detector = OverfittingDetector(baseline_sharpe=baseline_sharpe)

    context = DetectionContext(
        live_sharpe=live_sharpe,
        window_size=250,
        model_id="xgboost",
    )
    event = detector.evaluate(context)

    assert event is not None
    assert event.severity == "CRITICAL"
    assert event.detector_id == "D4_OVERFITTING"
    assert event.target == "xgboost"
    assert event.metric_value == 1.0
    assert event.threshold == 1.25 # 0.5 × 2.5

def test_d4_overfitting_detector_does_not_fire():
    """D4: Live Sharpe ≥ 0.5 × backtest → does NOT fire."""
    baseline_sharpe = 2.5
    live_sharpe = 1.5 # ratio = 0.6 ≥ 0.5
    detector = OverfittingDetector(baseline_sharpe=baseline_sharpe)

    context = DetectionContext(live_sharpe=live_sharpe, window_size=250,
        model_id="xgboost")
    event = detector.evaluate(context)

    assert event is None

def test_overall_health_min_model_dominates():
    """Overall health: min(model_health) weighted highest (30%)."""
    scores = HealthScores(
        model_health={"xgb": 95, "lstm": 60, "transformer": 90, "rl": 88},
        execution_quality=92,
        risk=90,
        broker_quality={"icmarkets": 95, "exness": 93},
        regime_confidence=88,
        overall=0, # computed below
    )
    overall = CEOSupervisor._aggregate(
        scores.model_health, scores.execution_quality,
        scores.risk, scores.broker_quality, scores.regime_confidence
    )
    # min_model = 60 (LSTM), min_broker = 93
    # 0.30*60 + 0.25*90 + 0.20*92 + 0.15*88 + 0.10*93
    # = 18.0 + 22.5 + 18.4 + 13.2 + 9.3 = 81.4
    assert overall == pytest.approx(81.4, abs=0.1)
    # LSTM's low score (60) drags overall below 85 (GREEN threshold)
    assert overall < 85 # would trigger YELLOW

```

Chapter 10 — Deployment Documentation

The CEO deploys as a single Python asyncio process on the same 4-vCPU VPS as the TITAN trading core. It requires PostgreSQL 15 (with TimescaleDB), Python 3.12, and 4 Python packages (asyncpg, numpy, nats-py, prometheus-client). Total deployment: 10 steps, ~30 minutes. The CEO runs as a systemd service, auto-restarts on failure, and integrates with the existing Prometheus + Grafana + PagerDuty stack.

10-Step Deployment Guide

```
# Step 1: Provision VPS (if not already running TITAN core)
# 4 vCPU, 8GB RAM, 100GB SSD, Ubuntu 22.04 LTS
# (CEO shares VPS with trading core – no separate hardware)

# Step 2: Install PostgreSQL 15 + TimescaleDB
sudo apt install postgresql-15 postgresql-15-timescaledb
sudo -u postgres psql -c "CREATE DATABASE titan_ceo;"
sudo -u postgres psql -d titan_ceo -c "CREATE EXTENSION timescaledb;"

# Step 3: Create schema (10 tables + hypertables + audit triggers)
psql -d titan_ceo -f /opt/titan/ceo/sql/schema.sql

# Step 4: Install Python 3.12 + dependencies
sudo apt install python3.12 python3.12-venv
python3.12 -m venv /opt/titan/ceo/venv
/opt/titan/ceo/venv/bin/pip install asyncpg numpy nats-py prometheus-client

# Step 5: Configure CEO
sudo mkdir -p /etc/titan
sudo cat > /etc/titan/ceo.yaml << 'EOF'
nats_url: "nats://localhost:4222"
db_dsn: "postgresql://titan@localhost/titan_ceo"
cycle_interval_s: 60
pagerduty_webhook: "" # optional – offline if empty
slack_webhook: "" # optional
models:
- xgboost
- lstm
- transformer
- rl
brokers:
- exness
- icmarkets
- pepperstone
- tickmill
- fp_markets
- fusion_markets
EOF

# Step 6: Install systemd service
sudo cat > /etc/systemd/system/titan-ceo.service << 'EOF'
[Unit]
Description=TITAN Meta AI CEO Supervisor
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=simple
User=titan
ExecStart=/opt/titan/ceo/venv/bin/python -m titan_ceo.supervisor
Restart=always
```

```
RestartSec=10
Environment=TITAN_CEO_CONFIG=/etc/titan/ceo.yaml

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable titan-ceo
sudo systemctl start titan-ceo

# Step 7: Import Grafana dashboard
# (8 panels: 6 health scores + system status + action log)
grafana-cli dashboards import /opt/titan/ceo/grafana/ceo-dashboard.json

# Step 8: Configure Prometheus scrape
# Add to prometheus.yml:
# - job_name: 'titan-ceo'
# scrape_interval: 15s
# static_configs:
# - targets: ['localhost:9101']

# Step 9: Configure PagerDuty (optional)
# Set webhook in /etc/titan/ceo.yaml
# P1 = RED status, P2 = YELLOW status

# Step 10: Smoke test
# Feed 100 synthetic trades via NATS, verify:
# - 6 score rows in system_health_scores
# - GREEN status
# - No control_actions fired
# - Cycle time < 100ms
python -m titan_ceo.smoke_test --trades 100
```

Chapter 11 — Operational Notes

The CEO is designed for 24/7 unattended operation. It auto-recovers from transient failures (NATS disconnect, PostgreSQL connection drop) via asyncio retry logic. If the CEO process crashes, systemd auto-restarts within 10 seconds. The CEO never blocks the trading core — all control actions are async (await `model_ctrl.set_influence()`), and the ensemble voter applies weight changes within 100ms of receiving them. The CEO is a governance layer, not a latency-critical path component.

Failure Modes

NATS disconnect: CEO buffers last-known scores, continues operating with stale data for up to 5 minutes. After 5 minutes, status degrades to YELLOW (stale data). **PostgreSQL disconnect:** CEO buffers scores in memory (up to 1000 cycles = ~16 hours), retries every 15 seconds. No data loss. **CEO process crash:** systemd restarts within 10s. On restart, CEO loads last-known status from DB, re-syncs rolling windows from trade ledger. **False positive (CEO disables healthy model):** Manual re-enable via CLI: `python -m titan_ceo.cli enable-model xgboost`. CEO logs every action with trigger reason for post-hoc analysis.

Monitoring the CEO Itself

The CEO exports 15 Prometheus metrics: `ceo_cycle_duration_seconds` (histogram), `ceo_current_status` (gauge: 0=GREEN/1=YELLOW/2=RED/3=PRESERVE), `ceo_overall_health` (gauge 0-100), `ceo_model_health` (gauge per model), `ceo_eqs`, `ceo_risk_score`, `ceo_broker_quality` (per broker), `ceo_regime_confidence`, `ceo_detectors_fired_total` (counter per detector), `ceo_actions_executed_total` (counter per action), `ceo_nats_events_ingested_total`, `ceo_db_write_latency_ms`, `ceo_rolling_window_size` (gauge per window). Alerting: `ceo_current_status == 2 (RED) for >60s` → P1 PagerDuty. `ceo_cycle_duration_seconds P99 > 200ms` → P2 (CEO is slow). `ceo_db_write_latency_ms P99 > 1000ms` → P2 (DB slow).

Integration with Existing TITAN Stack

The CEO integrates with 4 existing TITAN modules via well-defined interfaces: (1) M11 Hybrid AI Stack — CEO controls ensemble voter via `IModelController` (`set_influence`, `disable`, `enable`). The ensemble voter checks influence weights before each vote, applying them within 100ms. (2) M08 Risk Engine — CEO triggers risk mode changes via `IRiskController` (`set_mode`, `emergency_flatten`). The risk engine applies mode changes within 500ms. (3) M03 Execution Engine — CEO observes fills and latency via NATS subscription (read-only). No control. (4) M20 Observability — CEO exports Prometheus metrics that M20 scrapes. **No existing module requires modification** — the CEO observes via subscriptions and controls via interfaces. This is a drop-in addition.

Chapter 12 — Summary

The Meta AI CEO Supervisor (Module 18) is the runtime governance layer that the TITAN XAU AI system was missing. While the existing 5 validation frameworks (Backtest, Walk-Forward, Monte Carlo, Stress, Validator) gate pre-deployment, the CEO provides **continuous runtime governance** — monitoring all models, execution, risk, brokers, and regimes in real-time, detecting degradation before it becomes catastrophic, and taking automated corrective action. The CEO does NOT generate signals (enforced by validator test VT-001). It is a pure governance layer that observes, scores, and controls.

The CEO delivers on all requirements: 6 health scores (Model Health, EQS, Risk, BQS, Regime Confidence, Overall), 8 detectors (degradation, drift, instability, overfitting, exec deterioration, broker issues, regime failures, latency), 4 rolling windows (50/100/250/500 trades), 8 tracked metrics (WR/PF/Sharpe/Sortino/MDD/Recovery/Latency/Slippage), GREEN/YELLOW/RED status, 5 control actions (reduce/increase influence, disable model, emergency risk reduction, capital preservation). It operates under 4 hard constraints: no paid APIs, no external LLM, CPU optimized (<100ms/cycle), fully offline capable. The implementation is fully specified: 16 Python classes, 5 interfaces, 10 PostgreSQL tables, 145 tests (80 unit + 45 integration + 20 validator), and a 10-step deployment guide.

The CEO is the final piece of the TITAN governance architecture. With the CEO in place, the system has: (1) pre-deployment validation (5 frameworks), (2) runtime governance (CEO), (3) post-deployment monitoring (M20 Observability), and (4) quarterly re-validation (all 5 frameworks on live strategies). This is a complete, institutionally-rigorous governance lifecycle — the kind that separates world-class trading systems from retail bots. **The CEO watches the watchers.**